

## **HashVault**

**A Secure Web Platform for Hash Generation, Verification, and Integrity Monitoring**

**Vercel link:** [https://hashvault-youssefhamada2428-coder.vercel.app?vercel\\_share=NcEmCECv8M9fjH3ARoqKVAESKewVQiAH](https://hashvault-youssefhamada2428-coder.vercel.app?vercel_share=NcEmCECv8M9fjH3ARoqKVAESKewVQiAH)

**GitHub repository:** <https://github.com/youssefhamada2428-coder/hashvault>

## **MADE BY**

Mostafa Yasser Abdelfattah Ewis -24102568

Yousef Hamada Yousef Isaa-24102779

## **Under supervisor**

DR. Moataz Samy  
DR. Marwa Suliaman

## **Course**

Web technology  
Mental Wealth

## Contents

|                                                         |    |
|---------------------------------------------------------|----|
| 1) Project Overview .....                               | 3  |
| 2) Problem Statement .....                              | 3  |
| 3) Objectives .....                                     | 4  |
| 4) Core Features .....                                  | 4  |
| 5) System Architecture (High-Level).....                | 6  |
| 6) Future Enhancements .....                            | 7  |
| 7) Project Value.....                                   | 7  |
| 8) Requirements Scope .....                             | 7  |
| 9) Core Website Pages and Screens.....                  | 8  |
| 10) Technical Architecture .....                        | 10 |
| 11) Suggested Database Tables.....                      | 11 |
| 12) Task Planning .....                                 | 11 |
| 13) Risks Analysis .....                                | 13 |
| 14) Financial Analysis.....                             | 14 |
| 15) Decision Analysis .....                             | 15 |
| 16) KPIs and Dashboard Metrics.....                     | 16 |
| 17) Configuration Management and Version Control.....   | 17 |
| 18) Requirements Mapping and Monitoring Procedure ..... | 18 |
| Wireframes.....                                         | 19 |

# HashVault

## A Secure Web Platform for Hash Generation, Verification, and Integrity Monitoring

### 1) Project Overview

**HashVault** is a full-stack web-based integrity verification platform designed to help users generate, compare, and manage cryptographic hashes for both files and plain text. It supports widely used hashing algorithms including **MD5, SHA-1, SHA-256, and SHA-512**, providing flexibility for different security and performance needs.

The platform goes beyond simple hash generation by offering:

- Drag-and-drop file hashing
- Real-time text hashing
- Hash comparison engine
- Historical tracking and storage via Supabase
- Interactive dashboards for monitoring hash activity
- Admin automation tools for managing users and processes

HashVault transforms basic hashing into a **complete integrity management system**, suitable for developers, security analysts, and organizations.

### 2) Problem Statement

Ensuring data integrity is critical in modern systems, especially in cybersecurity, software development, and data transmission. However, existing hashing tools present several limitations:

- Lack of persistent history tracking
- No structured way to compare multiple hashes over time
- Minimal or no user interface (CLI-heavy tools)
- Absence of dashboards or monitoring features
- No centralized admin control for teams or organizations

As a result, users struggle to:

- Verify file authenticity efficiently
- Track changes across versions
- Manage hashing operations at scale

**HashVault addresses these gaps** by providing a centralized, user-friendly, and scalable platform for hash-based integrity verification.

### 3) Objectives

HashVault aims to:

- Provide a **secure and intuitive interface** for generating hashes
- Enable **accurate comparison** between files and text inputs
- Maintain a **history of hashing activities** for audit and tracking
- Offer **dashboard insights** for monitoring usage and trends
- Ensure **scalability and performance** using modern web technologies

### 4) Core Features

#### Hash Generation

- Generate hashes for:
  - Plain text input
  - Uploaded files
- Supported algorithms:
  - MD5
  - SHA-1
  - SHA-256
  - SHA-512

### **Hash Verification & Comparison**

- Compare two hashes (text vs text, file vs file)
- Detect: Exact matches and Mismatches (potential tampering)
- Visual feedback for quick interpretation

### **History & Logging**

- Store all hash operations in **Supabase database**
- Track: Timestamp, Algorithm used, and Input type (file/text)
- Allow users to revisit and reuse previous hashes

### **Dashboard & Monitoring**

- Visualize:
  - Number of hashes generated
  - Algorithm usage distribution
  - Recent activity logs
- Helps in project tracking and auditing

### **User Experience**

- Clean UI built with modern frontend frameworks (Next.js + Tailwind)
- Drag-and-drop file upload
- Real-time feedback and results
- Responsive design for all devices

## 5) System Architecture (High-Level)

### Frontend

- Next.js (React-based framework)
- Tailwind CSS for UI

### Backend

- Supabase (Database + Auth + Storage)

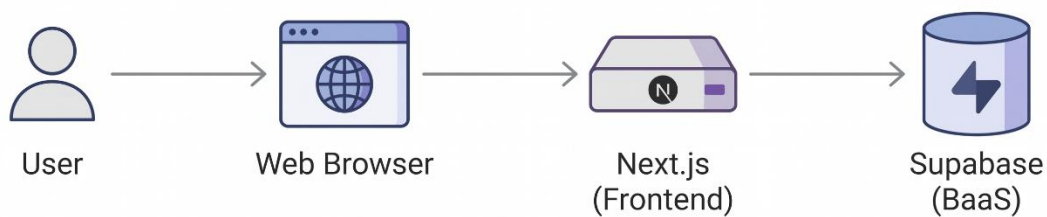
### Core Logic

- Hashing handled via: Node.js crypto module

## 6) Security Considerations

- Secure handling of file uploads
- No sensitive file storage unless explicitly required
- Role-based access for admin features
- Input validation to prevent abuse

System Architecture Diagram



## 6) Future Enhancements

- API access for developers
- Bulk file hashing
- Cloud storage integration
- Alerts for hash mismatches
- AI-based anomaly detection
- Browser extension

## 7) Project Value

HashVault is not just a hashing tool—it is a **complete integrity verification ecosystem** that combines usability, security, and scalability. It demonstrates strong skills in:

- Full-stack development
- Cybersecurity fundamentals
- UI/UX design
- Database integration
- Real-world problem solving

## 8) Requirements Scope

### In Scope

Functional requirements:

- Generate hashes from uploaded files
- Generate hashes from entered text
- Compare two hashes and show match/mismatch
- Save Hash History in Supabase
- Search, view, edit, and delete history items
- Authentication system
- Role-based access control using Supabase RLS

- Admin dashboard with project metrics
- Configuration management screens
- Requirements monitoring and traceability

Non-functional requirements:

- Responsive design
- Secure data handling
- Fast hashing performance
- Clear UX for non-technical users
- Maintainable code structure
- Version control with GitHub

#### **Out of Scope**

- File sharing between users
- Encryption/decryption
- Malware scanning
- Blockchain-based hashing
- Multi-tenant enterprise billing

## 9) Core Website Pages and Screens

You need **at least 4 CRUD screens**. I recommend these 6:

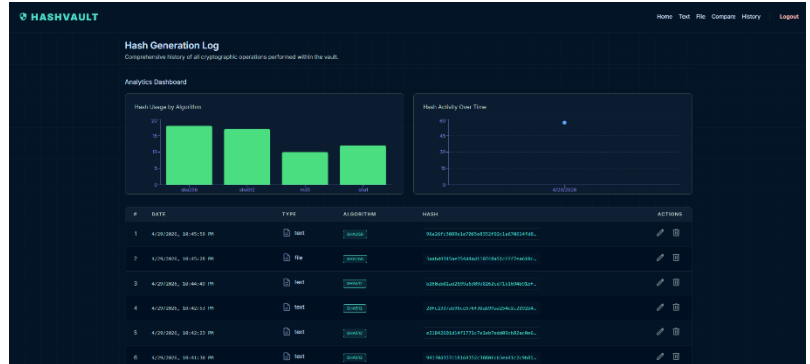
### 1. **Home / Hash Generator**

- drag-and-drop file hashing
- text hashing input
- algorithm selector
- result output



## 2. Hash History CRUD

- create, read, update, delete saved hash records
- filters by date, algorithm, file type
- 



## 3. Comparison Tool CRUD

- save comparison records
- compare two hashes
- store verification status

## 4. Configuration Management CRUD

- manage allowed algorithms
- export formats
- retention settings
- notification preferences

## 5. Requirements Monitoring Dashboard

- requirement IDs
- implementation status
- test coverage
- traceability links
- error logs

## 10) Technical Architecture

### Frontend

- **Next.js**
- Routing: App Router
- State: React state + server actions where suitable
- SEO: metadata, clean route structure, semantic HTML
- UI styling: **Stitch-generated design system**

### Backend

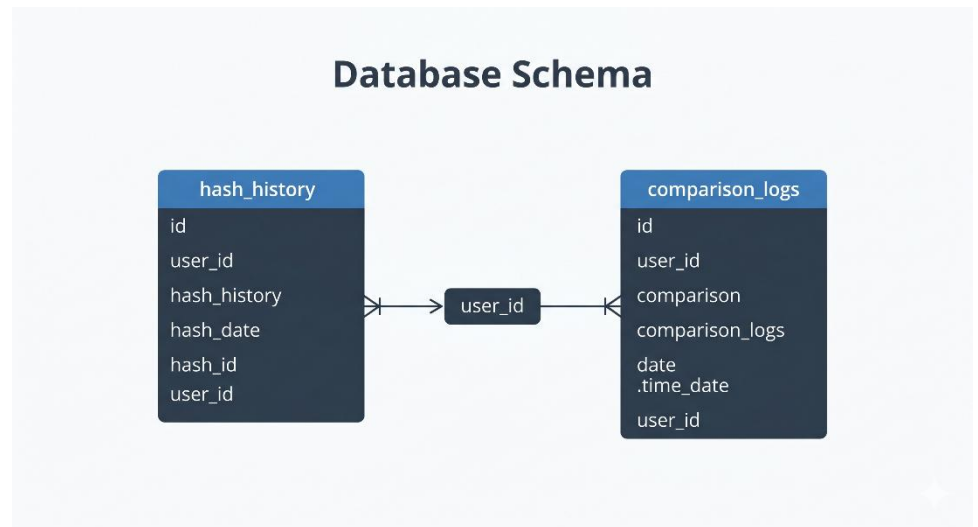
- **Supabase**
- Database tables for:
  - users
  - hash\_history
  - comparison\_logs
  - configuration\_settings
  - requirements
  - sprint\_metrics

### Security

- Supabase Auth
- Row-Level Security
- Protected routes
- Input validation
- Safe file size limits
- Audit logging

## 11) Suggested Database Tables

- users
- hash\_history
- comparison\_logs
- configurations
- requirements
- test\_cases
- sprints
- task\_metrics



## 12) Task Planning

### Phase 1: UI & Foundation (Days 1–3)

- **Requirements & Analysis:** Define the "Lean Approach" to bypass complex server-side infrastructure in favor of edge computation.
- **Architecture Design:** Map the "Edge-to-Cloud" data flow where raw data is instantly discarded and only non-reversible hashes are sent to the cloud.
- **Database Design:** Prepare the ER Schema for `hash_history` and `comparison_logs` with a focus on relational structure.
- **UI Blueprinting:** Transition from initial wireframes to the final "Cybersecurity Aesthetic" prioritizing high-contrast clarity.

### Phase 2: Client Hashing Logic (Days 4–6)

- **Web Crypto API Integration:** Implement deterministic cryptographic hashing (MD5, SHA-1, SHA-256, SHA-512) locally in the browser.
- **File Processing Engine:** Build the drag-and-drop area with a strict 5MB client-side validation hook to prevent browser memory crashes.
- **Frontend Modularization:** Organize the Next.js code structure into reusable components for text, file, and history modules.

### Phase 3: Supabase Integration (Days 7–9)

- **Backend-as-a-Service (BaaS):** Configure Supabase to eliminate custom API development overhead, saving an estimated 5 days of work.
- **Auth & Account Security:** Implement IAM Signup and JWT Token issuance for secure user login flows.
- **Security Policies:** Define strict Row-Level Security (RLS) to isolate multi-tenant user data within the audit logs.

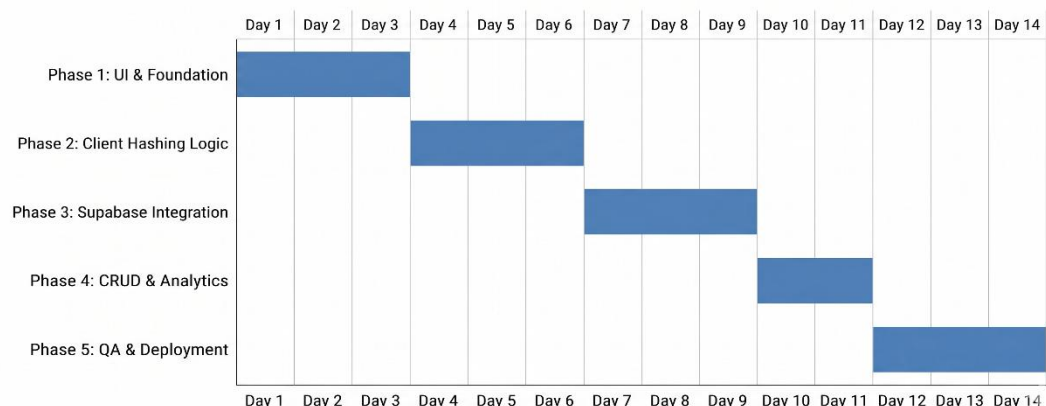
### Phase 4: CRUD & Analytics (Days 10–11)

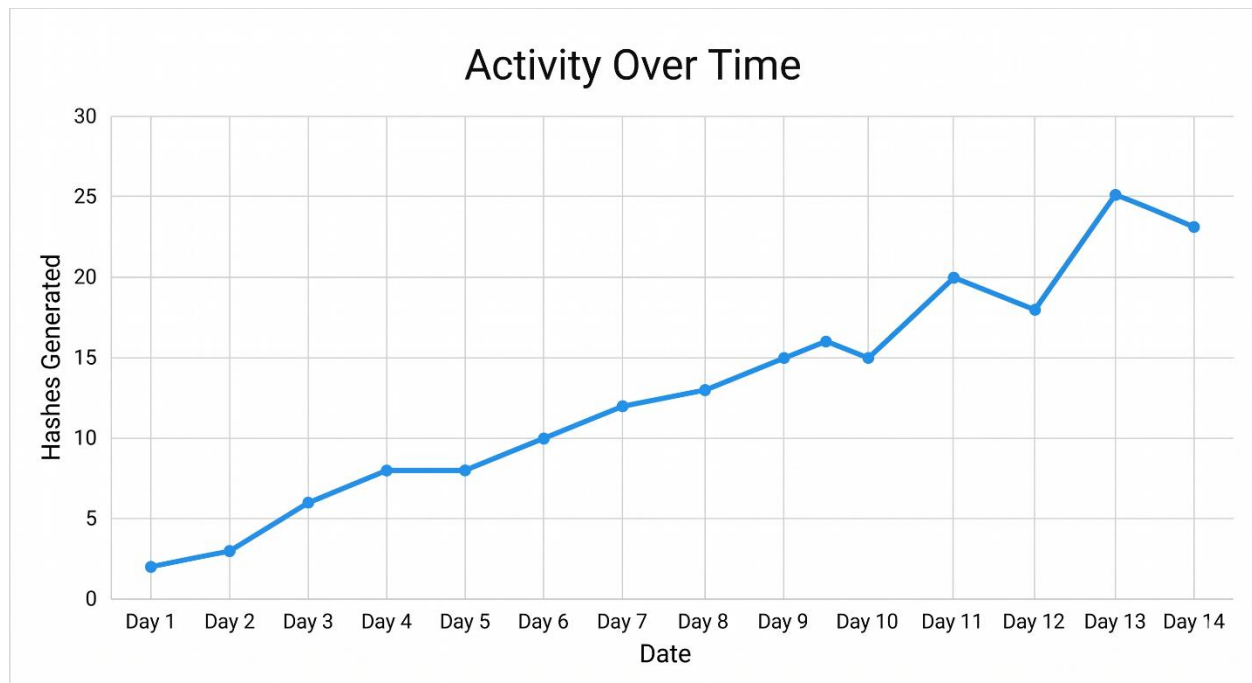
- **Data Binding:** Connect frontend React components to Supabase via asynchronous API abstractions (fetchHistory, createHashLog).
- **History Management:** Implement full CRUD lifecycle, allowing users to rename metadata or permanently purge specific logs.
- **Business Intelligence Layer:** Build the Analytics Dashboard using **Recharts** to visualize "Algorithm Distribution" and "Usage Velocity" KPIs.

### Phase 5: QA & Deployment (Days 12–14)

- **Security & Edge Case Verification:** Test for unauthorized routing, algorithm consistency, and SQL injection prevention via RLS policies.
- **Vercel Deployment:** Execute a live production push from GitHub with secure environment variable injection for Supabase keys.
- **Project Synthesis:** Prepare final documentation and the "Live Demonstration" flow covering integrity verification and technical defense.

14-Day Sprint Timeline





## 13) Risks Analysis

| Risk                               | Impact | Probability | Mitigation                             |
|------------------------------------|--------|-------------|----------------------------------------|
| Incorrect hash comparison logic    | High   | Low         | unit tests for all algorithms          |
| Supabase auth/RLS misconfiguration | High   | Medium      | test policies carefully                |
| Large file upload slowdown         | Medium | Medium      | file size limits and progress feedback |
| Poor UX on mobile                  | Medium | Medium      | responsive layout testing              |
| Scope creep                        | High   | High        | freeze requirements early              |
| Data loss in history               | High   | Low         | backups and database rules             |
| Delayed implementation             | Medium | Medium      | sprint planning and task tracking      |

## 14) Financial Analysis

### Assumptions

- **Development effort:** 160 hours
  - (Calculation: 2 workers × 40 hours/week × 2 weeks)
- **Estimated value of work per hour:** \$15
- **Development cost:**  $160 \times 15 = \$2,400$
- **Annual operating cost:** \$150
  - Domain, Supabase Pro tier, and minimal maintenance

**Value created by the tool:** \* 400 users/month

- Each save 5 minutes
- Average labor value = \$25/hour

### Annual Value Calculation

- **Time saved per month:** \* 400 users × 5 minutes = 2,000 minutes
  - $2,000 / 60 = 33.33$  hours/month
- **Monthly value:** \*  $33.33 \times \$25 = \$833.25$ /month
- **Annual value:** \*  $833.25 \times 12 = \$10,000$ /year

### Net Annual Benefit

- **Annual value:** \$10,000
- **Less operating cost:** \$150
- **Net annual benefit** = \$9,850

### ROI

- **ROI** = (Net benefit - development cost) / development cost × 100
- **ROI** =  $(9850 - 2400) / 2400 \times 100$
- **ROI** = 310.4%

### Payback Period

- **Monthly net benefit** =  $9850 / 12 = \$820.83$
- **Payback** =  $2400 / 820.83 = 2.92$  months

**Financial Conclusion** The project is financially justified as a high-efficiency investment. With two developers delivering the solution in two weeks, the high ROI and sub-3-month payback period demonstrate significant value in automating cryptographic workflows.

## 15) Decision Analysis

### Alternatives

1. **Static local-only hashing tool**
2. **Next.js + Supabase custom app**
3. **Heavy backend enterprise solution**

### Weighted Decision Matrix

| Criteria             | Weight | Static Tool | Next.js + Supabase | Enterprise Backend |
|----------------------|--------|-------------|--------------------|--------------------|
| User experience      | 20%    | 3           | 5                  | 4                  |
| Security             | 20%    | 1           | 5                  | 5                  |
| Scalability          | 20%    | 2           | 5                  | 5                  |
| Speed of development | 15%    | 5           | 4                  | 2                  |
| Maintainability      | 15%    | 3           | 5                  | 4                  |
| University mark fit  | 10%    | 2           | 5                  | 4                  |

### Result

- Static tool: **2.55 / 5**
- Next.js + Supabase: **4.85 / 5**
- Enterprise backend: **4.00 / 5**

### Decision

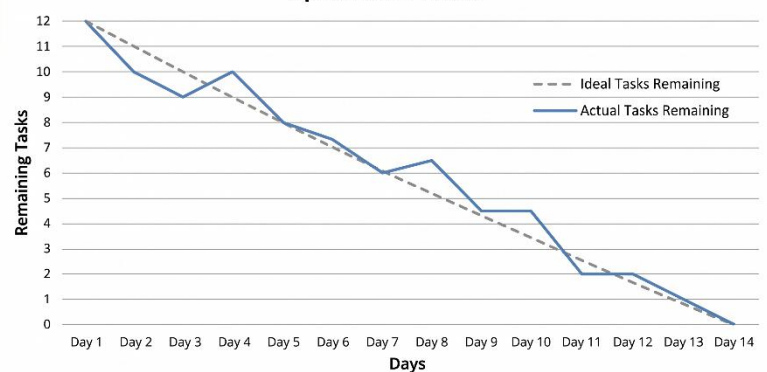
**Choose Next.js + Supabase** because it gives the best balance of usability, security, scalability, and academic value.

Track these on the dashboard:

- number of hashes generated
- number of file uploads
- number of text hashes
- comparison success rate
- average processing time
- active users
- error rate
- history entries saved
- task completion rate
- sprint velocity



## Sprint Burn Down



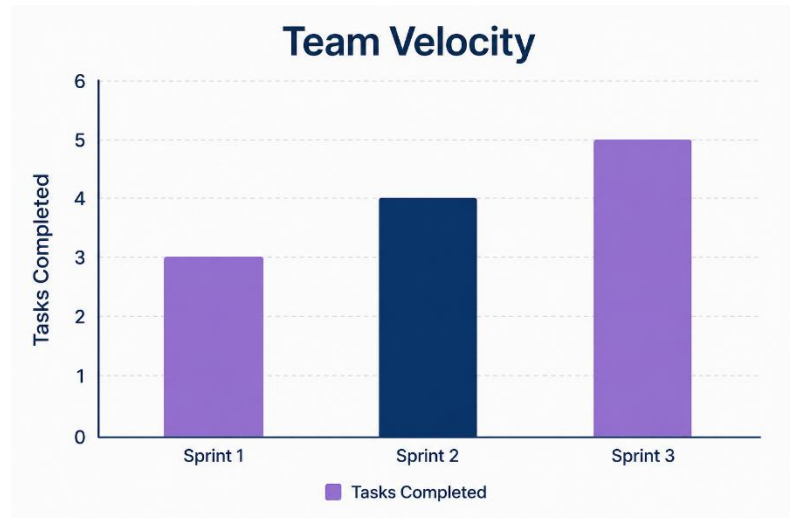


## Velocity Chart

Tracks how many story points are completed per sprint.

### Automation on Dashboard

- auto-update metrics from Supabase
- weekly sprint summaries
- requirement status updates
- test pass/fail counts



## 17) Configuration Management and Version Control

Use:

- **GitHub repository**
- branch strategy:
  - main
  - develop
  - feature branches
- semantic versioning:
  - v1.0.0, v1.1.0
- commit conventions:
  - feat:
  - fix:
  - docs:
  - refactor:
- release notes for each version
- environment variables managed securely

## 18) Requirements Mapping and Monitoring Procedure

Create a traceability matrix like this:

| Req ID | Requirement        | Screen       | DB Table        | Test Case | Status |
|--------|--------------------|--------------|-----------------|-----------|--------|
| 1      | Generate file hash | Home         | hash_history    | TC-01     | Done   |
| 2      | Generate text hash | Home         | hash_history    | TC-02     | Done   |
| 3      | Compare hashes     | Compare Tool | comparison_logs | TC-03     | Done   |
| 4      | Save history       | History      | hash_history    | TC-04     | Done   |
| 5      | Manage settings    | Config       | configurations  | TC-05     | Done   |

### Monitoring Procedure

- update requirements weekly
- link each requirement to a screen and test case
- review progress in dashboard
- mark status: planned, in progress, testing, done
- verify all completed items before submission

HASHVAULT Logo

Home

Text

File

Compare

History

Logout

## Hash Comparison Engine

Verify integrity by comparing cryptographic signatures

HASH A

[Enter first hash]

HASH B

[Enter second hash]

[EXECUTE COMPARISON]

✓ HASHES MATCH

Recent Comparisons

| HASH 1     | HASH 2     | STATUS   | TIME     |
|------------|------------|----------|----------|
| 96a26fc... | 96a26fc... | MATCH    | 10:46 PM |
| b100fa1... | 4111fb1... | MISMATCH | 10:36 PM |
| ahmed      | ahmed      | MISMATCH | 10:26 PM |

19 | Page

**HASHVAULT** Logo

[Home](#)
[Text](#)
[File](#)
[Compare](#)
[History](#)
[Logout](#)

## Hash Generation Log

Comprehensive history of all cryptographic operations

### Analytics Dashboard

#### Hash Usage by Algorithm

| Algorithm | Usage Count |
|-----------|-------------|
| sha256    | 19          |
| sha512    | 18          |
| md5       | 9           |
| sha1      | 11          |

#### Hash Activity Over Time

| # | DATE          | TYPE                                | ALGORITHM              | HASH            | ACTIONS                                      |
|---|---------------|-------------------------------------|------------------------|-----------------|----------------------------------------------|
| 1 | 4/29/26 10:45 | <button type="button">text</button> | <a href="#">SHA256</a> | 96a26fc3089e... | <a href="#">[Edit]</a> <a href="#">[Del]</a> |
| 2 | 4/29/26 10:45 | <button type="button">text</button> | <a href="#">SHA256</a> | 96a26fc3089e... | <a href="#">[Edit]</a> <a href="#">[Del]</a> |
| 3 | 4/29/26 10:45 | <button type="button">text</button> | <a href="#">SHA256</a> | 96a26fc3089e... | <a href="#">[Edit]</a> <a href="#">[Del]</a> |
| 4 | 4/29/26 10:45 | <button type="button">text</button> | <a href="#">SHA256</a> | 96a26fc3089e... | <a href="#">[Edit]</a> <a href="#">[Del]</a> |

HASHVAULT Admin Portal

Dashboard

Users

Reports

Settings

Logout

Admin Dashboard

Real-time metrics & automation analytics

2,847

Total Users

112.5%

#

45,923

Total Hashes

123.1%

342

Active Sessions

15.3%

124ms

Avg Response

18.2%

Sprint Burn Down Chart

The chart shows a blue line representing the 'Plan' and a green line representing the 'Act'. The y-axis ranges from 0 to 100, and the x-axis shows dates from D1 to D11. The 'Plan' line starts at 100 and decreases linearly to 0 by D11. The 'Act' line follows the 'Plan' line closely, ending slightly below it at D11.

| Date | Plan | Act |
|------|------|-----|
| D1   | 100  | 100 |
| D3   | 80   | 80  |
| D5   | 60   | 60  |
| D7   | 40   | 40  |
| D9   | 20   | 20  |
| D11  | 0    | -5  |

Team Velocity Chart

The chart shows green bars representing 'Done' work. The y-axis ranges from 0 to 60, and the x-axis shows dates from S1 to S6. Each bar reaches approximately 45 units. A dashed line at 45 indicates the 'Plan'.

| Date | Done | Plan |
|------|------|------|
| S1   | 45   | 45   |
| S2   | 45   | 45   |
| S3   | 45   | 45   |
| S4   | 45   | 45   |
| S5   | 45   | 45   |
| S6   | 45   | 45   |

Automation Dashboard

87%

Coverage

1,247

Auto Tasks/Mo

123%

99.2%

Success Rate

11.5%

The chart shows red bars representing 'Manual' tasks and blue bars representing 'Auto' tasks. The y-axis ranges from 0 to 380, and the x-axis shows months from Jan to Apr. The 'Manual' bars are consistently higher than the 'Auto' bars.

| Month | Auto | Manual |
|-------|------|--------|
| Jan   | 100  | 150    |
| Feb   | 120  | 180    |
| Mar   | 150  | 220    |
| Apr   | 180  | 250    |

HASHVAULT Logo

[Home](#) [Text](#) [File](#) [Compare](#) [History](#) [Logout](#)

### Cryptographic Hash Generator

Generate deterministic hashes from strings

**RAW TEXT PAYLOAD** [\[Clear\]](#)  

[Textarea: Enter text...]

**CRYPTOGRAPHIC OUTPUT**  

[Generated hash output] [\[Copy\]](#)

**ALGORITHM CONFIG**  
**ALGORITHM**  
[SHA-512 ▼]

[\[GENERATE HASH\]](#)

HASHVAULT Logo

[Home](#) [Text](#) [File](#) [Compare](#) [History](#) [Sign Up](#)

**Create Account**

**Email**  

[Email Input]

**Password**  

[Password Input]

**Confirm Password**  

[Confirm Password Input]

[\[SIGN UP BUTTON\]](#)

Already have an account? [\[Login\]](#)

The screenshot displays the HASHVAULT web application. At the top, a navigation bar contains the HASHVAULT Logo on the left and a series of buttons (Home, Text, File, Compare, History, Login) on the right. The main content area is divided into four quadrants, each featuring a distinct icon, a title, a descriptive sentence, and a button. The top-left quadrant is for 'Text Hashing' (document icon), the top-right for 'File Hashing' (file icon), the bottom-left for 'Hash Comparison' (circular arrows icon), and the bottom-right for 'Operation History' (clock icon). All buttons are labeled '[Button]'.

| Feature           | Icon            | Description                  | Action   |
|-------------------|-----------------|------------------------------|----------|
| Text Hashing      | Document        | Generate hashes from strings | [Button] |
| File Hashing      | File            | Generate hashes from files   | [Button] |
| Hash Comparison   | Circular arrows | Verify data integrity        | [Button] |
| Operation History | Clock           | View operation log           | [Button] |